

ThaiSLM — file structure (v7)

Everything a person continuing the project needs: what every path holds, where the deployable model is, how to run the service, how to rebuild data and models. Last updated with the v7 release ([result_reporting/result_v7.md](#)).

1. Top level

path	what it is	in git?
<code>main.py</code>	the only CLI entry point — <code>python main.py</code> lists every command (<code>infer · test · data · prep · extract · harvest · cloud · train-seq · eval · export-onnx · pretrained</code>)	✓
<code>docker-compose.yml</code>	web demo / service — <code>docker compose up --build</code> → <code>http://localhost:8080</code>	✓
<code>.dockerignore</code>	keeps the Docker build context to code + the MediaPipe model (data and models never enter images)	✓
<code>requirements.txt</code>	Python 3.10 environment for training / development (<code>conda env hugging</code>)	✓
<code>README.md</code>	start-here guide	✓
<code>lab.ipynb</code>	post-training lab: data, latent space, tagger, segmentation, results, self-made sentence tests, face channel, production examples (generated by <code>scripts/make_lab.py</code>)	✓
<code>.env</code>	secrets and local settings (OpenAI key, GCP project / bucket, AWS SSO profile) — never committed	✗
<code>.gitignore</code> , <code>.gitattributes</code>	what goes to git; model weights via Git LFS (<code>*.pt</code> , <code>*.npz</code> , <code>*.onnx</code> , <code>*.task</code> under <code>models/</code>)	✓

2. `models/` — the deployable model (load this to use ThaiSLM)

file	content
<code>mediapipe/holistic_landmarker.task</code>	the pose / hand / face extractor (MediaPipe Holistic, tasks 1.0.1) — identical file in Python and in the browser
<code>encoder.pt</code> + <code>mt5_config.json</code>	sign encoder (Uni-Sign ST-GCN + mT5 encoder + head), PyTorch
<code>tagger.pt</code>	frame tagger (BiGRU): non-sign / sign-begin / sign-inside
<code>span_head.npz</code>	one-pass spotting head (NumPy)
<code>bank.npz</code> , <code>concepts.json</code>	vocabulary bank (one embedding per labelled clip) and concept names
<code>vocab_conversation.json</code>	daily-conversation vocabulary (conversation mode)
<code>null_proto.npy</code> , <code>freq_prior.npy</code>	"not a sign" prototype, Thai word-frequency prior
<code>spotting.json</code> , <code>segment_calibration.json</code>	decoder and calibration settings per vocabulary
<code>grammar.json</code> , <code>roles.json</code> , <code>roles_language.json</code>	TSL word order + word roles (re-ranker) · roles for composing Thai sentences (daily words set by category)
<code>fingerspell.npz</code> , <code>letters_bank.npz</code>	fingerspelling recogniser (3 averaged NumPy MLPs, A–Z + $n-\emptyset$ + 7 vowels) and the letter bank (encoder embeddings); settings in <code>spotting.json</code> "spell"
<code>face_template.npz</code>	mean face shape for the Standard Schema
<code>config.json</code>	provenance (runs, sizes, version)
<code>onnx/encoder.onnx</code> , <code>onnx/tagger.onnx</code> (+ <code>encoder.int8.onnx</code> if parity allows), <code>onnx/parity.json</code>	production graphs for ONNX Runtime
<code>README.md</code>	how to load / extend the model

Use it: `SignPipeline(backend="onnx", device="cpu", vocab="conversation").run("video.mp4")` (see `models/README.md`), or the web API.

3. modules/ — the library

module	role	used at inference?
mediapipe_pose.py	MediaPipe Holistic extraction (video file) and browser-landmark parsing → raw arrays → Standard Schema	✓
schema.py	Standard Schema (133-slot layout, 25 fps, face template, hand-gap filling), handedness canonicalisation, activity / sign span	✓
parts.py	Standard Schema → the four part streams the encoder reads (NumPy)	✓
runtime.py	ONNX Runtime / PyTorch back-ends with one NumPy interface; vocabulary Bank ; window tables; exact re-scoring	✓
spanhead.py	one-pass spotting head: NumPy inference + distillation training	✓
segment.py	kinematic features, windows, temporal resampling, tagger decoding, boundary metrics (NumPy)	✓
decode.py	semi-Markov decoder, word priors, metrics, calibration (logistic), transition / repeat post-processing	✓
pipeline.py	SignPipeline : everything end to end (+ result.json / subtitle.srt / timeline.png)	✓
face.py	face channel: blendshapes → question / negation / emotion (calibrated on neutral dictionary faces)	✓
fingerspell.py	fingerspelling: hand-shape window features, NumPy MLP, letter events	✓
language.py	evidence-guarded Thai sentence (OpenAI or rules)	✓
grammar.py	TSL word-order patterns (TSL51) + optional re-ranker	✓
tts.py	Thai speech (MMS-TTS VITS offline; edge / OpenAI optional)	✓ (tts service)
encoder.py , unisign.py	sign encoder (PyTorch), training data sampler (hard negatives, cross-extractor positives, webcam augmentation), losses	training / torch back-end
sequence.py	tagger model, sentence synthesis, forced alignment, the local sequence stage	training
lexicon.py	gloss → lemma → concept (rules, synonym groups, data_test references)	data prep
identity.py	signer identity from faces (DINOv2) — data prep only	data prep
utils.py , env.py	paths (THAISLM_MODELS , THAISLM_DATA , THAISLM_ARTIFACTS overrides), logging, .env loader	✓

4. `scripts/` — pipeline steps (all reachable through `main.py`)

script	what it does
<code>data_prep.py</code>	raw data → <code>data_prep/</code> (video list, ingest of landmark-only sources, signers, lexicon, manifest + QC, training shards, SCHEMA.md)
<code>extract_mp.py</code>	MediaPipe on every source video (local CPU, resumable, two processes can share the work)
<code>harvest_words.py</code>	YouTube single-word lessons for daily words: discover → download (<code>data_test</code> duplicate check) → videos → signers (faces, <code>data_test</code> look-alike exclusion) → label (span + consistency)
<code>eval.py</code>	build <code>models/</code> , grammar, decoder tuning (<code>spot</code>), isolated evaluation (<code>isolated</code>), <code>data_test</code> (<code>infer</code>), self-made sentence videos (<code>selftest</code>), vocab exports
<code>export_onnx.py</code>	<code>models/onnx/*.onnx</code> + parity check
<code>infer.py</code>	translate one video (CLI)
<code>make_lab.py</code>	writes / executes <code>lab.ipynb</code>
<code>harvest_letters.py</code>	alphabet lessons → letter holds (<code>agent_data/en_alpha</code> , <code>th_alpha</code>)
<code>train_fingerspell.py</code>	trains the fingerspelling recogniser + letter bank (CPU)
<code>tune_spelling.py</code>	letter settings on tune data, report on held-out data
<code>study_clips.py</code>	reference clips of each word / letter of a sentence (e.g. <code>data_test/jack_test/</code>)
<code>report_figures.py</code>	figures for the report (<code>result_reporting/figures_v7/</code>)
<code>md_to_pdf.py</code>	Markdown report → PDF (headless Chrome)
<code>s3_sync.py</code>	S3 data lake pull (<code>python main.py data pull ...</code>)
<code>pretrained.py</code>	rebuild <code>artifacts/pretrained/</code> (Uni-Sign base weights from Hugging Face)
<code>collect/</code>	the original raw-data harvester (TTRS / YouTube, v4) — kept to rebuild <code>raw_data</code>

5. `GCP/` — cloud GPU (Vertex AI)

path	content
<code>GCP/script/vertex.py</code>	<code>upload-code</code> · <code>upload-data</code> · <code>upload-models</code> · <code>launch</code> (same job in several regions, first to start wins, the rest are cancelled while pending) · <code>wait</code> · <code>status</code> · <code>cancel</code> · <code>cleanup</code> (empty the bucket, delete every job, check that no VM / disk / IP / endpoint bills)
<code>GCP/script/job_entry.py</code>	what runs inside the container (<code>train_encoder</code> ; <code>sequence</code> runs locally via <code>python main.py train-seq</code>)
<code>GCP/script/gcp_common.py</code>	<code>.env</code> → project / bucket / machine profiles, cost estimate
<code>GCP/script/follow_logs.py</code>	stream a running job's log lines
<code>GCP/script/requirements_job.txt</code>	extra pip packages inside the PyTorch container
<code>GCP/result/*.json</code>	one record per job: machine, region, queue / run minutes, estimated cost (v7 jobs; earlier records archived)

6. webapp/ — the web demo (micro-services)

path	service
webapp/web/	Next.js 16 front end (components/SignStudio.tsx camera / live mode / upload, components/ResultCard.tsx , lib/holistic.ts MediaPipe in the browser, lib/api.ts)
webapp/api/	sign-api (FastAPI + ONNX Runtime) — app.py , Dockerfile , requirements.txt
webapp/tts/	tts (FastAPI + MMS-TTS Thai)
webapp/gateway/nginx.conf	gateway routes / → web, /api/ → sign-api, /tts/ → tts
webapp/README.md	how to run, API reference, scaling notes

7. artifacts/ — training outputs that are not the deployable model

path	content
pretrained/	Uni-Sign base (unisign_wlas1_enc.pt , not in git — python main.py pretrained) + mt5_config.json
runs/<run>/	one folder per training run: train_history.json , metrics_*.json , summary.json , job.log ; the local sequence run (seq-v7/) : tagger, span head, alignment, sequence_metrics.json (per-clip arrays are not in git)
eval/bank_eval.npz	the train-only bank (honest evaluation: held-out signers are never in it)
eval/grammar_tune.json	TSL word-order patterns learned from tune templates only (decoder tuning)
eval/emb_v6_encoder_mp.npz	v6-encoder embeddings of the same MediaPipe queries (v6 column of the isolated comparison; not in git)
reports/*.json	every evaluation: isolated.json , spotting.json , infer_final.json (data_test), selftest.json , phrases.json (real phrases), natural_tuning.json , nmm_validation.json (face cues)
runs/	only the v7 runs are kept: enc-v7-europe-west4 (encoder) and seq-v7 (span head, taggers)

8. Data (not in git — S3 `s3://dsi490-lake-signdata/`, local mirror `cloud_s3/`)

path	content
<code>cloud_s3/raw_data/</code>	raw videos / landmark releases (TTRS, th_sl, TSL-ONE-S, TSL51, YouTube)
<code>cloud_s3/data_prep/</code>	Standard Schema: <code>pose_mp/</code> + <code>pose_mp_raw/</code> (v7 MediaPipe), <code>pose/</code> (v6 RTMW + landmark-only sources), <code>manifest/</code> , <code>params/</code> (<code>video_items.csv</code> , signer clusters, <code>tsl51_alignment_v6.json</code> = transition source for the shards ...), <code>qc/</code> , <code>shards/</code> (training input), <code>SCHEMA.md</code>
<code>cloud_s3/agent_data/en_alpha/</code> , <code>th_alpha/</code>	fingerspelling lessons, landmarks, letter labels (<code>items.csv</code>), README per folder
<code>cloud_s3/agent_data/youtube_words/</code>	v7 harvest: <code>videos/</code> + <code>meta/</code> (yt metadata), <code>candidates.csv</code> , <code>phrases.csv</code> , <code>download_status.csv</code> , <code>videos.csv</code> , <code>signers.csv</code> , <code>items.csv</code> (the annotations: word, concept, span, signer, role, status + reason) , <code>qc/</code> (signer contact sheet, identity report)
<code>data_test/</code>	5 real-world test videos (never used for training or tuning)
<code>cache/</code>	scratch, not in git: MediaPipe of <code>data_test</code> (<code>data_test_mp/</code>), self-made test videos (<code>selftest/</code>), <code>legacy_v5/</code> (v5 intermediates <code>prep ingest still reads</code>), <code>thsl_face_emb.npz</code> , <code>daily_depth_v6.csv</code>

9. `vocab/` and `result_reporting/`

path	content
<code>vocab/daily_conversation.json</code>	the ~340-word daily-conversation target list (by category)
<code>vocab/daily_conversation_depth.csv</code>	signers per daily word in v7
<code>vocab/missing_word_strategy.json</code>	how words with no Thai data are covered (Thai synonym / composition / face / fingerspelling — no foreign signs)
<code>vocab/vocab.csv</code>	every concept: instances, signers, datasets, role, in the conversation vocabulary
<code>result_reporting/result_vN.md/.pdf</code>	reports per version (v5, v6, v7)
<code>result_reporting/architecture_vN</code> , <code>latent_space_vN</code>	architecture / representation reports
<code>result_reporting/figures_v7/</code>	report figures (v5 / v6 figures live inside their PDFs)
<code>result_reporting/inference/<vocab>/<video>/</code>	<code>data_test</code> outputs (<code>result.json</code> , <code>subtitle.srt</code> , <code>timeline.png</code>)
<code>result_reporting/file_structure.md/.pdf</code>	this document

10. Common tasks

```
# run the service
docker compose up --build                                # http://localhost:8080
docker compose up --build --scale sign-api=3             # more API replicas behind the gateway
THAISLM_MODELS_DIR=./other_models docker compose up     # serve another model folder

# use the model from Python / CLI
python main.py infer --video clip.mp4 --vocab conversation --tts
python main.py test                                     # data_test end-to-end

# add a word / a signer without retraining: extract → embed → append to models/bank.npz (models/README.md)

# rebuild everything
python main.py data pull data_prep                      # or rebuild: prep pose_items → extract → prep
ingest/lexicon/manifest
python main.py prep shards
python main.py cloud upload-code --tag v7 && python main.py cloud upload-data && python main.py cloud upload-models
python main.py cloud launch --tag v7 --stage train_encoder --hours 1.2 --args "--steps 3000 --eval_every 500"
python main.py train-seq --encoder_run artifacts/runs/<run>
python main.py eval build --run <run> --seq seq-v7 && python main.py eval grammar && python main.py eval spot --seq
seq-v7
python main.py export-onnx && python main.py test && python scripts/make_lab.py --execute
python main.py cloud cleanup                            # ALWAYS after a GPU round
```

11. Workspace clean-up (v7)

v5 / v6 leftovers that nothing reads any more were moved out of the repository to `D:\SLM-labs\_thaislm_v7_cleanup\` (old training runs, v5/v6 vocab exports, v5/v6 report figures, debug caches, old GCP job records; $\approx$ 6.4 GB). Delete that folder when you no longer need it. The v5 / v6 reports (`.md` + `.pdf`) are kept; their figures are embedded in the PDFs.